## COLLEGE OF ENGINEERING, GUINDY

### DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY


## MINI PROJECT ABSTRACT

# SMART CV ANALYSER USING C

**SUBJECT:** DATA STRUCTURES USING C

**TOPIC:** SMART CV ANALYSER


**SUBMITTED BY:**

| S. NO. | NAME | REGISTER NUMBER |
|---|---|---|
| 1. | DHARSHINI R | 2025115063 |
| 2. | YUTHIKA K | 2025115031 |
| 3. | KALAIVANI E | 2025115027 |

# SMART CV ANALYSER USING C

## 1. Explanation

The Smart CV Analyser is a console-based mini project developed using the C programming language to demonstrate the practical use of data structures in analysing candidate profiles. The program accepts user input such as a candidate's name, education, career objective, experience, project description, and skills. It allows users to enter job requirements and compares them with the candidate's profile using simple rule-based matching. Based on the entered criteria, the program calculates a match score, identifies missing skills, and displays suggestions for improvement. Candidate profiles are stored in a linked list, pending CVs are managed using a queue, and recent analysis results are maintained using a stack. The program is menu-driven and runs in a GCC-compatible environment without a frontend, backend, or file handling.

## 2. Data Structures and Concepts Used

- Structures (struct) and typedef
- Singly linked list
- Queue using linked nodes
- Stack using linked nodes
- Dynamic memory allocation using malloc() and free()
- String handling and case-insensitive skill comparison
- Searching and rule-based matching
- Functions, arrays, and control statements

## 3. Data Structures and Their Purpose

**Structures:** Store related candidate information, queue entries, and analysis history in organised records.

**Singly Linked List:** Stores candidate profiles dynamically and allows the program to maintain multiple CV records during execution.

**Queue:** Holds candidate IDs waiting for analysis. The FIFO (First In, First Out) principle ensures that the earliest queued CV is analysed first.

**Stack:** Stores analysis results. The LIFO (Last In, First Out) principle makes the most recent result appear first in the history.

**Dynamic Memory Allocation:** Uses malloc() to create nodes when required and free() to release memory when the program exits.

## 4. Main Modules

- **Candidate Profile Entry:** Accepts the candidate's name, education, objective, experience, project details, and skills.
- **CV Display:** Displays all candidate profiles stored in the linked list.
- **Queue Management:** Adds candidate IDs to a queue and displays pending CVs.
- **Job Requirement Entry:** Accepts a job role, education keyword, minimum experience, required skills, and project requirement.
- **CV Analysis and Score Calculation:** Compares entered skills and criteria, calculates a score out of 100, and classifies the result as a strong, moderate, or low match.
- **Improvement Suggestions:** Lists missing required skills and highlights profile details that may need improvement.
- **Analysis History:** Displays previous analysis results, with the latest result shown first.

## 5. Real-World Applications

- Can be used as a basic demonstration of candidate profile screening for a specified job role.
- Helps students understand how job requirements can be compared with candidate skills.
- Demonstrates queue-based processing of multiple CV profiles.
- Shows how data structures can organise and retrieve information in a recruitment-related application.

## 6. Advantages

- Simple menu-driven program that accepts user input.
- Runs using standard C and GCC without external libraries.
- Demonstrates linked lists, queues, stacks, structures, and dynamic memory allocation in one project.
- Automatically calculates a rule-based match score and identifies missing skills.
- Does not require a frontend, backend, or file handling.

## 7. Conclusion

The Smart CV Analyser demonstrates the practical application of fundamental data structures using C. It supports candidate profile entry, queue-based CV processing, job requirement matching, score calculation, improvement suggestions, and analysis history. By combining a linked list, queue, and stack in a menu-driven program, the project illustrates how data structures can be applied to a recruitment-related problem. The score is based only on the criteria entered by the user; it is a rule-based result and not an AI model or an automated hiring decision.